

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**

Controle de versão com Git e GitHub

Branches e merging

Aula 2

Código da aula: [DADOS]ANO1C1B4S27A2

Controle de versão
com Git e GitHub

Mapa da Unidade 8 Componente 1

semana

25

Branches no Git

Você está aqui!

Branches e merging

semana

27

semana

20

Fundamentos
de Git

semana

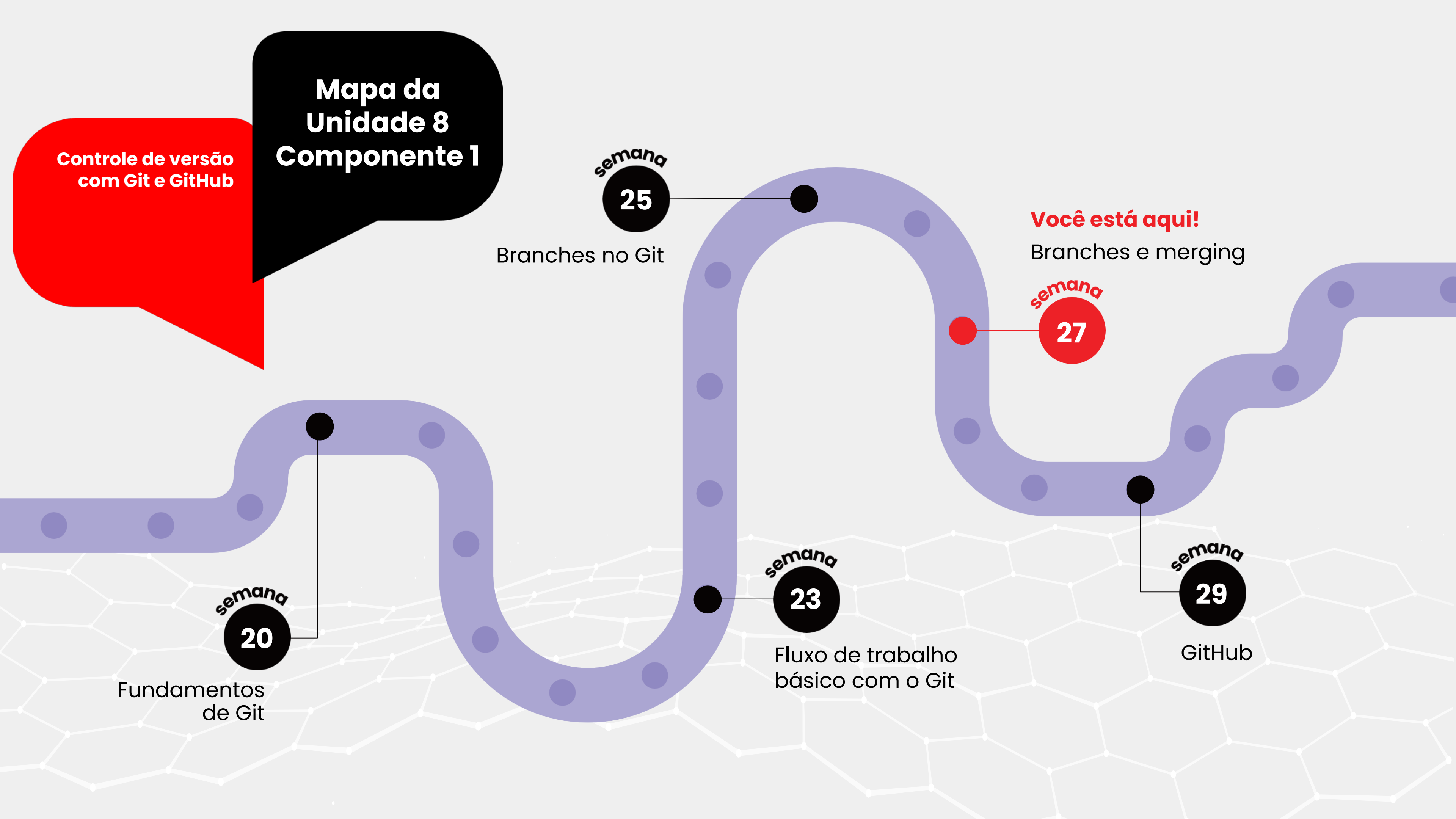
23

Fluxo de trabalho
básico com o Git

semana

29

GitHub



**Controle de versão
com Git e GitHub**

Mapa da Unidade 8 Componente 1

Você está aqui!

Branches e merging

Aula 2

Código da aula: [DADOS]ANO1C1B4S27A2

27



Objetivos da Aula

- Introduzir e utilizar ferramentas de desenvolvimento de software como Git e GitHub.



Recursos Didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou à internet.



Duração da Aula

50 minutos.



Competências Técnicas

- Usar ferramentas de desenvolvimento de software como Git e GitHub;
- Compreender e dominar técnicas de manipulação de dados.



Competências Socioemocionais

- Analisar informações de forma crítica e tomar decisões; avaliar a qualidade dos dados e a eficácia dos modelos e das técnicas de análise;
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados; trabalhar em equipes multifuncionais colaborando com colegas, gestores e clientes.

Construindo
o **conceito**

Rebasing

Rebase é a operação de mover ou combinar uma sequência de commits para uma nova base. Isso permite uma linha de histórico mais limpa e linear, evitando commits de merge desnecessários.

Comandos de rebase:

- Iniciar um rebase: **git rebase <branch>**
- Interromper um rebase: **git rebase --abort**
- Continuar um rebase: **git rebase --continue**

Construindo
o **conceito**

Diferença entre rebase e merge

Rebase	Merge
Reaplica os commits de uma branch sobre outra, criando um histórico linear sem commits de merge.	Combina commits de diferentes branches preservando o histórico de commits de ambas.

Construindo
o **conceito**

Cherry-picking

Cherry-pick é útil quando se quer aplicar uma correção ou uma pequena mudança específica sem precisar mesclar uma branch inteira.

Comandos de cherry-pick:

Cherry-pick de um commit: **git cherry-pick <commit-hash>**

Construindo
o **conceito**

Stashing

Stash é a operação de guardar temporariamente mudanças não commitadas, permitindo que um desenvolvedor mude de branch ou faça outras operações sem perder o trabalho em andamento.

Quando usar stash:

Stash é útil quando se precisa mudar de branch rapidamente ou quando se deseja limpar o diretório de trabalho sem perder as mudanças não commitadas.

Construindo
o **conceito**

Merging estratégico

Existem várias estratégias de merge que podem ser utilizadas dependendo da necessidade do projeto:

- ▶ **Squash merge:** combina vários commits em um único commit. Utilizado para manter um histórico de commits mais limpo.
- ▶ **Octopus merge:** permite mesclar múltiplas branches em uma única operação de merge. Útil em projetos com muitas branches de pequenas mudanças.

Construindo
o **conceito**

Merging estratégico

Cada estratégia de merge tem seus benefícios e desafios:

- ▶ **Squash merge** é útil para manter um histórico limpo, mas pode esconder a granularidade dos commits.
- ▶ **Octopus merge** facilita a integração de múltiplas branches, mas pode ser complicado de gerenciar em grandes projetos.

Construindo o **conceito**

Exemplo: rebasing

1. Inicializar um repositório Git:

```
!mkdir rebase_example_project  
%cd rebase_example_project  
!git init
```

2. Criar um arquivo Python com uma função simples:

```
%%writefile main.py  
def main():  
    print("Hello from the main branch")  
  
main()
```

Elaborado especialmente para o curso com o prompt de comando.

Construindo
o **conceito**

Exemplo: rebasing

3. Commitar mudanças:

```
!git add main.py  
!git commit -m "Add main function"
```

4. Criar a branch main:

```
!git branch -M main
```

Elaborado especialmente para o curso com o prompt de comando.

Construindo o **conceito**

Exemplo: rebasing

5. Criar e mudar para uma nova branch:

```
!git checkout -b feature/new-feature
```

6. Modificar o arquivo Python na nova branch:

```
%%writefile main.py
def main():
    print("Hello from the feature branch")

main()
```

Elaborado especialmente para o curso com o prompt de comando.

Construindo o **conceito**

Exemplo: rebasing

7. Commitar mudanças na nova branch:

```
!git add main.py  
!git commit -m "Update main function in feature branch"
```

8. Voltar para a branch principal (main) e fazer outra mudança:

```
!git checkout main
```

```
%%writefile main.py  
def main():  
    print("Hello from the main branch - updated")  
  
main()
```

Construindo o **conceito**

Exemplo: rebasing

9. Commitar mudanças na branch principal:

```
!git add main.py  
!git commit -m "Update main function in main branch"
```

10. Rebase na branch principal:

```
!git checkout feature/new-feature  
!git rebase main
```

Elaborado especialmente para o curso com o prompt de comando.



Colocando em **prática**

Exercício: stashing

Implemente o código no prompt de comando (terminal) e descreva o que cada comando está fazendo. Ao final, encaminhe em documento-texto pelo AVA.

```
# Inicializar um repositório Git
!mkdir stash_example_project
%cd stash_example_project
!git init

# Criar o arquivo README.md e fazer o commit inicial
with open("README.md", "w") as file:
    file.write("# Projeto de Exemplo\n\nEste é um projeto de exemplo para demonstrar o uso de stash no Git.\n")

!git add README.md
!git commit -m "Initial commit"

# Criar a branch main
!git branch -M main

# Criar uma nova branch para desenvolvimento
!git checkout -b feature/stash-example

# Desenvolver uma nova funcionalidade
with open("new_feature.py", "w") as file:
    file.write("# Implementação de uma nova funcionalidade

def new_feature():
    print("This is a new feature!")

# Exemplo de uso
new_feature()
")
```



20 minutos



Em duplas



Documento-texto



Colocando em **prática**

Exercício: stashing

Implemente o código no prompt de comando (terminal) e descreva o que cada comando está fazendo. Ao final, encaminhe em documento-texto pelo AVA.

```
# Criar um stash das mudanças não commitadas
!git add new_feature.py
!git stash

# Verificar o estado do repositório (não deve haver mudanças não commitadas)
!git status

# Mudar para a branch principal (main) e fazer mudanças
!git checkout main

with open("README.md", "w") as file:
    file.write("# Projeto de Exemplo")

Este é um projeto de exemplo para demonstrar o uso de stash no Git.

Atualização do README para a branch principal.
"""

!git add README.md
!git commit -m "Update README"

# Voltar para a branch de desenvolvimento e aplicar o stash
!git checkout feature/stash-example
!git stash apply

# Commitar as mudanças stashed
!git add new_feature.py
!git commit -m "Add new feature after stashing"
```



20 minutos

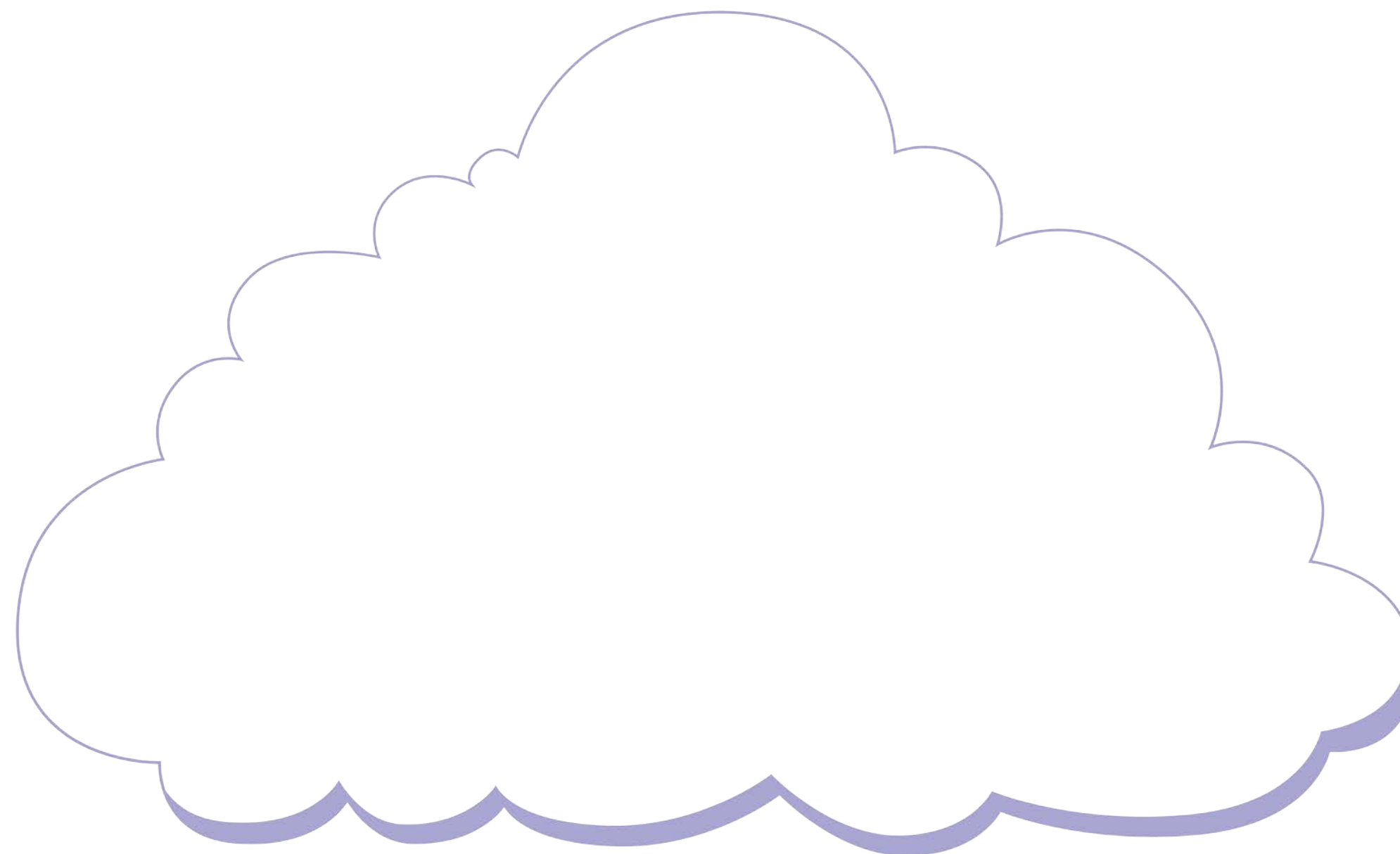


Em duplas



Documento-texto

Nuvem de palavras



© Getty Images

O que nós
**aprendemos
hoje?**



© Getty Images

O que nós
**aprendemos
hoje?**

Então ficamos assim...

- 1** Rebase em Git move ou combina uma sequência de commits para uma nova base, criando um histórico linear. Comandos básicos incluem **git rebase**, **git rebase --abort**, e **git rebase --continue**.
- 2** Cherry-pick aplica um commit específico de uma branch em outra, útil para aplicar correções sem mesclar uma branch inteira. O comando básico é **git cherry-pick <commit-hash>**.
- 3** Stashing guarda temporariamente mudanças não commitadas, permitindo mudanças de branch sem perder progresso. Comandos incluem **git stash**, **git stash list**, **git stash apply**, e **git stash drop**. Estratégias de merge, como squash e octopus, ajudam a gerenciar o histórico de commits.

Saiba mais

Você pode aprender muito mais sobre Git e GitHub!

No link abaixo, você encontra um curso onde é possível aprender a compartilhar e colaborar em projetos, criar portfólios e muito mais! Vale a pena conferir!

ALURA. *Git e GitHub: compartilhando e colaborando em projetos*. Disponível em: <https://www.alura.com.br/curso-online-git-github-compartilhando-colaborando-projetos>. Acesso em: 23 jul. 2024.

Referências da aula

BELL, P.; BEER, B. *Introdução ao GitHub*: um guia que não é técnico. São Paulo: Novatec, 2015.

GITHUB DOCS. *Documentação de introdução ao GitHub*, [s.d.]. Disponível em: <https://docs.github.com/pt/get-started>. Acesso em: 23 jul. 2024.

Identidade visual: imagens © Getty Images

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**